Walmart Sales Data Analysis

SQL / PostgreSQL Exploratory Analysis Project Dataset

Project Overview

This project explores three years (2010–2012) of weekly sales data across 45 Walmart stores using PostgreSQL. The goal is to understand sales trends over time — at the yearly, monthly, and weekly level — and identify patterns such as seasonality, peak sales periods, and year-over-year performance.

Dataset columns:

Column	Description
Store	Store number (1–45)
Date	Week-ending date
Weekly_Sales	Sales for that store, that week
Holiday_Flag	1 if the week includes a major holiday, else 0
Temperature	Average regional temperature that week
Fuel_Price	Regional fuel price that week
CPI	Consumer Price Index for the region
Unemployment	Regional unemployment rate

Query 1: Rank each year by total sales -- (highest total sales = rank 1)

```
-- Rank each year by total sales using a window function -- (highest total sales = rank 1)

 SELECT year, yearly_sales,
        RANK() OVER (ORDER BY yearly_sales DESC) AS rank_year_sales
FROM ( SELECT  EXTRACT(YEAR FROM date) AS year,
               SUM(weekly_sales) AS yearly_sales
       FROM walmart_sales_data_45stores
       GROUP BY EXTRACT(YEAR FROM date)
) AS yearly_totals
ORDER BY yearly_sales DESC;
```

	year numeric	yearly_sales numeric	rank_year_sales bigint
1	2011	2448200007.35	1
2	2010	2288886120.41	2
3	2012	2000132859.35	3

Insight

Sales performance is not evenly split across the three years. Ranking total annual sales shows a clear top year, a middle year, and a bottom year, which suggests a broader trend

```
-- top 5 and bottom 5 stores by total sales overall
(SELECT store, SUM(weekly_sales) AS total_sales, 'Top 5' AS category
 FROM walmart_sales_data_45stores
 GROUP BY store
 ORDER BY total_sales DESC  LIMIT 5)
UNION ALL
(SELECT store, SUM(weekly_sales) AS total_sales, 'Bottom 5' AS category
 FROM walmart_sales_data_45stores
 GROUP BY store
 ORDER BY total_sales ASC   LIMIT 5);
```

	store integer	total_sales numeric	category text
1	20	301397792.46	Top 5
2	4	299543953.38	Top 5
3	14	288999911.34	Top 5
4	13	286517703.80	Top 5
5	2	275382440.98	Top 5
6	33	37160221.96	Bottom 5
7	44	43293087.84	Bottom 5
8	5	45475688.90	Bottom 5
9	36	53412214.97	Bottom 5
10	38	55159626.42	Bottom 5

Insight

Sales performance varies dramatically across the 45 stores. The top-performing store (Store 20) generated approximately $301.4M in total sales, roughly eight times more than the lowest-performing store (Store 33) at $37.2M. The top 5 stores are tightly clustered between $275M and $301M, suggesting a consistent group of high-volume "flagship" locations rather than a single outlier. In contrast, the bottom 5 stores range more widely from $37M to $55M, but remain in a clearly distinct tier from the top performers. This wide gap points to structural differences between stores — such as size, location, or regional demand — rather than simple week-to-week variance, and suggests store segmentation could be a valuable next step for deeper analysis.

Query 2: Daily Sales Aggregation (All Stores Combined)

```
-- Aggregate weekly sales by date,
-- showing total sales and record count per date across all 45 stores
SELECT date,
       SUM(weekly_sales) AS sum_weekly_45stores,
       COUNT(weekly_sales) AS total_45stores
FROM walmart_sales_data_45stores
GROUP BY date
ORDER BY date;
```

	date date	sum_weekly_45stores numeric	total_45stores bigint
1	2010-02-05	49750740.50	45
2	2010-02-12	48336677.63	45
3	2010-02-19	48276993.78	45
4	2010-02-26	43968571.13	45
5	2010-03-05	46871470.30	45
6	2010-03-12	45925396.51	45
7	2010-03-19	44988974.64	45
8	2010-03-26	44133961.05	45
9	2010-04-02	50423831.26	45
10	2010-04-09	47365290.44	45
11	2010-04-16	45183667.08	45
12	2010-04-23	44734452.56	45
13	2010-04-30	43705126.71	45
14	2010-05-07	48503243.52	45

Insight

This is the foundation for all later analysis — it collapses the 45 individual store rows for each week into a single combined sales figure per date. The record count column (total_45stores) is a useful sanity check: if it's consistently 45 for every date, it confirms the dataset is complete with no missing store-week records.

Query 3: Highest Sales Week Per Month

```
-- Highest sales week per month
SELECT DISTINCT ON (year_month)
       year_month,
       date,
       sum_weekly_45stores
FROM (
    SELECT date,
           SUM(weekly_sales) AS sum_weekly_45stores,
           TO_CHAR(date, 'YYYY-MM') AS year_month
    FROM walmart_sales_data_45stores
    GROUP BY date
) sub
ORDER BY year_month, sum_weekly_45stores DESC;
```

	year_month text	date date	sum_weekly_45stores numeric
1	2010-02	2010-02-05	49750740.50
2	2010-03	2010-03-05	46871470.30
3	2010-04	2010-04-02	50423831.26
4	2010-05	2010-05-07	48503243.52
5	2010-06	2010-06-04	50188543.12
6	2010-07	2010-07-02	48917484.50
7	2010-08	2010-08-06	48204999.12
8	2010-09	2010-09-03	47194257.61
9	2010-10	2010-10-08	45102974.23
10	2010-11	2010-11-26	65821003.24
11	2010-12	2010-12-24	80931415.60
12	2011-01	2011-01-07	42775787.77
13	2011-02	2011-02-18	48716164.12
14	2011-03	2011-03-04	46980603.74
15	2011-04	2011-04-22	48676692.06
16	2011-05	2011-05-06	46861958.29
17	2011-06	2011-06-03	48771994.18

Insight

Looking at the peak week within each month highlights seasonality at a finer grain than annual totals alone. December months stand out sharply from the rest of the year, consistent with holiday shopping (Black Friday and pre-Christmas weeks), while most other months show comparatively modest peak weeks.

Query 4: Highest & Lowest Single Sales Week Per Year (Combined)

```
-- Combine each year's highest and lowest single sales week into one table
-- Highest single sales week in each year (2010, 2011, 2012)
SELECT year, date, sum_weekly_45stores, 'Highest Week in the Year' AS week_type
FROM (
    SELECT DISTINCT ON (year)
       year,
       date,
       sum_weekly_45stores
   FROM (
       SELECT date,
           SUM(weekly_sales) AS sum_weekly_45stores,
           EXTRACT(YEAR FROM date) AS year
    FROM walmart_sales_data_45stores
    GROUP BY date
   ) sub
   ORDER BY year, sum_weekly_45stores DESC
) highest_week_each_year
UNION ALL  --combine both lowest and highest result in one table
-- lowest single sales week in each year (2010, 2011, 2012)
SELECT year, date, sum_weekly_45stores, 'Lowest Week in the Year' AS week_type
FROM (
    SELECT DISTINCT ON (year)
       year,
       date,
       sum_weekly_45stores
   FROM (
       SELECT date,
           SUM(weekly_sales) AS sum_weekly_45stores,
           EXTRACT(YEAR FROM date) AS year
    FROM walmart_sales_data_45stores
    GROUP BY date
   ) sub
   ORDER BY year, sum_weekly_45stores
) highest_week_each_year
ORDER BY year, sum_weekly_45stores DESC;
```

	year numeric	date date	sum_weekly_45stores numeric	week_type text
1	2010	2010-12-24	80931415.60	Highest Week in the Year
2	2010	2010-12-31	40432519.00	Lowest Week in the Year
3	2011	2011-12-23	76998241.31	Highest Week in the Year
4	2011	2011-01-28	39599852.99	Lowest Week in the Year
5	2012	2012-04-06	53502315.87	Highest Week in the Year
6	2012	2012-01-27	39834974.67	Lowest Week in the Year

Insight

For all three years, the highest week (2010-12-24, 2011-12-23, 2012-04-06) is far above the lowest week of the same year, and the gap between a year's best and worst week is large relative to typical weekly sales. Interestingly, 2012's peak week fell in April rather than December, which stands out from 2010 and 2011 and may be worth investigating further (e.g. a major promotion or missing December data for that year).

Query 5: Highest & Lowest Weekly Sales Per Month (Value Only)

```
-- For each year/month, find the highest and lowest
-- weekly total sales (across all 45 stores) recorded in that month
SELECT
    EXTRACT(YEAR FROM date) AS year,
    EXTRACT(MONTH FROM date) AS month,
    MAX(sum_weekly_45stores) AS highest_week,
    MIN(sum_weekly_45stores) AS lowest_week
FROM (
    SELECT date,
           SUM(weekly_sales) AS sum_weekly_45stores
    FROM walmart_sales_data_45stores
    GROUP BY date
) sub
GROUP BY EXTRACT(YEAR FROM date), EXTRACT(MONTH FROM date)
ORDER BY year, month;
```

	year numeric	month numeric	highest_week numeric	lowest_week numeric
1	2010	2	49750740.50	43968571.13
2	2010	3	46871470.30	44133961.05
3	2010	4	50423831.26	43705126.71
4	2010	5	48503243.52	45120108.06
5	2010	6	50188543.12	46609036.29
6	2010	7	48917484.50	44630363.42
7	2010	8	48204999.12	45909740.44
8	2010	9	47194257.61	41358514.41
9	2010	10	45102974.23	42239875.87
10	2010	11	65821003.24	45125584.18
11	2010	12	80931415.60	40432519.00
12	2011	1	42775787.77	39599852.99
13	2011	2	48716164.12	44125859.84
14	2011	3	46980603.74	42876199.18
15	2011	4	48676692.06	43458991.19
16	2011	5	46861958.29	44046598.01
17	2011	6	48771994.18	45884094.58
18	2011	7	47859263.78	43683274.28
19	2011	8	48015466.97	46249569.21
20	2011	9	46763227.53	42195830.81
21	2011	10	47211688.36	44374820.30
22	2011	11	66593605.26	46438980.56
23	2011	12	76998241.31	46042461.04
24	2012	1	44955421.95	39834974.67
25	2012	2	50197056.96	45771506.57
26	2012	3	47480454.11	44993794.45

Insight

This view complements Query 3 by showing the spread (highest vs. lowest week) within every month, not just the single best week. Months with a wide gap between their highest and lowest week indicate more volatile sales within that month, while months with a narrow gap suggest more consistent week-to-week performance.

Query 6: Average weekly sales: holiday vs. non-holiday weeks Compare average sales: holiday vs non-holiday weeks, with % difference

```
--Average weekly sales: holiday vs. non-holiday weeks
SELECT
    CASE WHEN holiday_flag = 1 THEN 'Holiday Week' ELSE 'Non-Holiday Week' END AS week_type,
    ROUND(AVG(weekly_sales), 2) AS avg_weekly_sales

FROM walmart_sales_data_45stores
GROUP BY holiday_flag;
```

	week_type text	avg_weekly_sales numeric
1	Non-Holiday Week	1041256.38
2	Holiday Week	1122887.89

```
-- Compare average sales: holiday vs non-holiday weeks, with % difference
SELECT
    ROUND(AVG(CASE WHEN holiday_flag = 0 THEN weekly_sales END), 2) AS avg_non_holiday,
    ROUND(AVG(CASE WHEN holiday_flag = 1 THEN weekly_sales END), 2) AS avg_holiday,
    ROUND(
        (AVG(CASE WHEN holiday_flag = 1 THEN weekly_sales END)
         - AVG(CASE WHEN holiday_flag = 0 THEN weekly_sales END))
        / AVG(CASE WHEN holiday_flag = 0 THEN weekly_sales END) * 100
    , 2) AS pct_difference
FROM walmart_sales_data_45stores;
```

	avg_non_holiday numeric	avg_holiday numeric	pct_difference numeric
1	1041256.38	1122887.89	7.84

Insight

Holiday weeks average about $1,122,887.89 in sales, compared to $1,041,256.38 for non-holiday weeks — roughly 7.8% higher. This confirms holiday weeks reliably drive a meaningful sales lift across stores, reinforcing the seasonal pattern already seen in the December peak weeks.

Query 7: Largest drop from a non-holiday week to the following holiday week

```
--Largest drop from a non-holiday week to the following holiday week
WITH weekly AS (
    SELECT store, date, weekly_sales, holiday_flag,
           LAG(weekly_sales) OVER (PARTITION BY store ORDER BY date) AS prev_sales,
           LAG(holiday_flag) OVER (PARTITION BY store ORDER BY date) AS prev_holiday_flag
    FROM walmart_sales_data_45stores
)
SELECT store, date AS holiday_week, prev_sales AS prior_week_sales,
       weekly_sales AS holiday_week_sales,
       (prev_sales - weekly_sales) AS sales_drop
FROM weekly
WHERE holiday_flag = 1 AND prev_holiday_flag = 0
ORDER BY sales_drop DESC
LIMIT 10;
```

	store integer	holiday_week date	prior_week_sales numeric	holiday_week_sales numeric	sales_drop numeric
1	14	2010-12-31	3818686.45	1623716.46	2194969.99
2	10	2010-12-31	3749057.69	1707298.14	2041759.55
3	20	2010-12-31	3766687.43	1799737.79	1966949.64
4	13	2010-12-31	3595903.2	1675292	1920611.2
5	4	2010-12-31	3526713.39	1794868.74	1731844.65
6	2	2010-12-31	3436007.68	1750434.55	1685573.13
7	4	2011-12-30	3676388.98	2007105.86	1669283.12
8	27	2010-12-31	3078162.08	1440963	1637199.08
9	13	2011-12-30	3556766.03	1969056.91	1587709.12
10	23	2010-12-31	2734277.1	1169773.85	1564503.25

Insight

All top 10 drops occurred around **Dec 31** (New Year's week) or late-Dec 30, and every case shows the same pattern: the prior week's sales (~$2.7M–$3.8M, the Christmas rush) collapse to a much lower holiday-week figure (~$1.2M–$2M) — drops of roughly $1.6M–$2.2M per store. This isn't holidays hurting sales in general; it's a post-Christmas comedown, where the massive pre-Christmas spike simply can't be sustained into the New Year's week itself. Store 14 saw the single largest drop (~$2.19M), but the pattern is broadly consistent across the top 10, suggesting this is a predictable seasonal effect rather than a store-specific issue.

Query 8: 4-week rolling average per store

```
--4-week rolling average per store
SELECT store, date, weekly_sales,
       ROUND(AVG(weekly_sales) OVER (
           PARTITION BY store ORDER BY date
           ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
       ), 2) AS rolling_4wk_avg
FROM walmart_sales_data_45stores
ORDER BY store, date;
```

	store integer	date date	weekly_sales numeric	rolling_4wk_avg numeric
1	1	2010-02-05	1643690.9	1643690.90
2	1	2010-02-12	1641957.44	1642824.17
3	1	2010-02-19	1611968.17	1632538.84
4	1	2010-02-26	1409727.59	1576836.03
5	1	2010-03-05	1554806.68	1554614.97
6	1	2010-03-12	1439541.59	1504011.01
7	1	2010-03-19	1472515.79	1469147.91
8	1	2010-03-26	1404429.92	1467823.50
9	1	2010-04-02	1594968.28	1477863.90
10	1	2010-04-09	1545418.53	1504333.13
11	1	2010-04-16	1466058.28	1502718.75
12	1	2010-04-23	1391256.12	1499425.30
13	1	2010-04-30	1425100.71	1456958.41
14	1	2010-05-07	1603955.12	1471592.56
15	1	2010-05-14	1494251.5	1478640.86
16	1	2010-05-21	1399662.07	1480742.35
17	1	2010-05-28	1432069.95	1482484.66
18	1	2010-06-04	1615524.71	1485377.06

Insight

The rolling average is doing exactly what it's meant to — smoothing out the week-to-week noise in Store 1's sales. For example, weekly sales bounce around quite a bit (e.g., $1,643,690.90 → $1,409,727.59 → $1,594,968.28), but the 4-week rolling average moves much more gradually ($1,643,690.90 → $1,576,836.03 → $1,477,863.90), making the underlying trend easier to see. This is useful for spotting genuine momentum (sustained rises or declines) rather than reacting to a single unusually high or low week — and it's a good foundation for later comparing whether a store's rolling trend is climbing, flat, or declining heading into a holiday period.